
ACCRA TECHNICAL UNIVERSITY
Faculty of Engineering
Department of Electrical/Electronic Engineering

BEE 208

Control Statements in C++

Practical Lab Activity Set 2

Course Code	BEE 208
Lab Topic	Control Statements in C++
Question Set	Set 2 – 20 Questions
Faculty	Engineering
Department	Electrical/Electronic Engineering
Question Types	Conditional, Switch, For, While, Do-While, Break, Continue
Student Deliverables	Algorithm Pseudocode Flowchart Test Results

Marking Guide Per Question

Component	Marks
Algorithm	2
Pseudocode	2
Flowchart	2
Test Results and Output Evidence	2

TOTAL	8
-------	---



TABLE OF CONTENTS

Question 1: Power Factor Status Checker	—	Control: <i>if...else if...else</i>
Question 2: Cable Temperature Warning System	—	Control: <i>if...else</i>
Question 3: Motor Starting Current Checker	—	Control: <i>if...else</i>
Question 4: Fuse Selection Menu	—	Control: <i>switch</i>
Question 5: Measurement Unit Conversion Menu	—	Control: <i>switch</i>
Question 6: Transformer Ratio Calculator Menu	—	Control: <i>switch</i>
Question 7: Total Power of Lighting Points	—	Control: <i>for loop</i>
Question 8: Average Current of Three-Phase Readings	—	Control: <i>for loop</i>
Question 9: Count Passed Component Tests	—	Control: <i>for loop with if...else</i>
Question 10: Calculate Total Energy for Appliances	—	Control: <i>for loop</i>
Question 11: Stop Motor Test When Temperature Is Unsafe	—	Control: <i>while loop</i>
Question 12: Solar Panel Voltage Tracker	—	Control: <i>while loop</i>
Question 13: Earth Resistance Test	—	Control: <i>while loop</i>
Question 14: Laboratory Login Attempt	—	Control: <i>while loop with a counter</i>
Question 15: Repeated Ohm's Law Calculator	—	Control: <i>do...while loop</i>
Question 16: Repeated DC Power Calculator	—	Control: <i>do...while loop</i>
Question 17: Stop Insulation Test Using Break	—	Control: <i>for loop with break</i>
Question 18: Stop Load Test When Limit Is Reached	—	Control: <i>for loop with break</i>
Question 19: Skip Faulty Sensor Readings	—	Control: <i>for loop with continue</i>
Question 20: Skip Negative Power Readings	—	Control: <i>for loop with continue</i>

QUESTION 1: Power Factor Status Checker

Required Control Statement: if...else if...else

1. ALGORITHM

1. START the programme.
2. PROMPT the user to enter the power factor of the electrical load.
3. READ and STORE the power factor value into a variable (e.g., pf).
4. CHECK: If pf is less than 0.00 OR greater than 1.00:
 - a. DISPLAY: 'Invalid power factor.'
5. ELSE IF pf is between 0.00 and 0.50 (inclusive):
 - a. DISPLAY: 'Poor power factor.'
6. ELSE IF pf is between 0.51 and 0.80 (inclusive):
 - a. DISPLAY: 'Fair power factor.'
7. ELSE IF pf is between 0.81 and 0.95 (inclusive):
 - a. DISPLAY: 'Good power factor.'
8. ELSE IF pf is between 0.96 and 1.00 (inclusive):
 - a. DISPLAY: 'Excellent power factor.'
9. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY 'Enter power factor of the electrical load: '
INPUT pf
IF (pf < 0.00 OR pf > 1.00) THEN
DISPLAY 'Invalid power factor.'
ELSE IF (pf >= 0.00 AND pf <= 0.50) THEN
DISPLAY 'Poor power factor.'
ELSE IF (pf >= 0.51 AND pf <= 0.80) THEN
DISPLAY 'Fair power factor.'
ELSE IF (pf >= 0.81 AND pf <= 0.95) THEN
DISPLAY 'Good power factor.'
ELSE
DISPLAY 'Excellent power factor.'
END IF
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
/INPUT pf (power factor)/
|
v
1?> --YES--> [DISPLAY 'Invalid power factor'] --> [STOP]
| NO
v
--YES--> [DISPLAY 'Poor power factor'] --> [STOP]
| NO
v
--YES--> [DISPLAY 'Fair power factor'] --> [STOP]
| NO
v
--YES--> [DISPLAY 'Good power factor'] --> [STOP]
| NO
v
[DISPLAY 'Excellent power factor']
|
v
[STOP]
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Valid Input (Good Range)

INPUT: Enter power factor of the electrical load: 0.88

OUTPUT: Power Factor Status: Good power factor.

Test Case 2 – Invalid Input (Out of Range)

INPUT: Enter power factor of the electrical load: 1.25

OUTPUT: Power Factor Status: Invalid power factor.

QUESTION 2: Cable Temperature Warning System

Required Control Statement: if...else

1. ALGORITHM

1. START the programme.
2. PROMPT the user to enter the cable temperature in degrees Celsius.
3. READ and STORE the temperature value into a variable (e.g., temp).
4. CHECK: If temp is greater than 70°C:
 - a. DISPLAY: 'Cable overheating detected.'
5. ELSE:
 - a. DISPLAY: 'Cable temperature is within safe range.'
6. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY 'Enter the cable temperature (°C): '
INPUT temp
IF (temp > 70) THEN
DISPLAY 'Cable overheating detected.'
ELSE
DISPLAY 'Cable temperature is within safe range.'
END IF
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
/INPUT temp (degrees Celsius)/
|
v
70°C?>
|YES |NO
v v
[DISPLAY 'Cable [DISPLAY 'Cable temperature
overheating detected.']] is within safe range.']]
| |
+-----+-----+
|
v
[STOP]
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Overheating Condition

INPUT: Enter the cable temperature (°C): 85

OUTPUT: Cable overheating detected.

Test Case 2 – Safe Condition

INPUT: Enter the cable temperature (°C): 55

OUTPUT: Cable temperature is within safe range.

QUESTION 3: Motor Starting Current Checker

Required Control Statement: if...else

1. ALGORITHM

1. START the programme.
2. PROMPT the user to enter the rated current of the motor (in Amperes).
3. READ and STORE rated current into a variable (e.g., ratedCurrent).
4. PROMPT the user to enter the starting current of the motor (in Amperes).
5. READ and STORE starting current into a variable (e.g., startCurrent).
6. CHECK: If startCurrent is greater than ($3 \times \text{ratedCurrent}$):
 - a. DISPLAY: 'High starting current. Use proper motor starter.'
7. ELSE:
 - a. DISPLAY: 'Starting current is acceptable.'
8. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY 'Enter rated current of motor (A): '
INPUT ratedCurrent
DISPLAY 'Enter starting current of motor (A): '
INPUT startCurrent
IF (startCurrent > 3 * ratedCurrent) THEN
DISPLAY 'High starting current. Use proper motor starter.'
ELSE
DISPLAY 'Starting current is acceptable.'
END IF
END
```


3. FLOWCHART TEXT-MAP

```

[START]
|
v
/INPUT ratedCurrent/
|
v
/INPUT startCurrent/
|
v
3 x ratedCurrent?>
|YES |NO
v v
[DISPLAY 'High starting [DISPLAY 'Starting current
current. Use motor is acceptable.']]
starter.']]
| |
+-----+-----+
|
v
[STOP]

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 - High Starting Current

INPUT: Enter rated current of motor (A): 10 Enter starting current of motor (A): 45

OUTPUT: High starting current. Use proper motor starter.

Test Case 2 - Acceptable Starting Current

INPUT: Enter rated current of motor (A): 20 Enter starting current of motor (A): 50

OUTPUT: Starting current is acceptable.

QUESTION 4: Fuse Selection Menu

Required Control Statement: switch

1. ALGORITHM

1. START the programme.
2. DISPLAY the fuse rating menu:
 1. 5 A Fuse
 2. 10 A Fuse
 3. 13 A Fuse
 4. 20 A Fuse
 5. 32 A Fuse
3. PROMPT the user to enter their menu choice.
4. READ and STORE the choice into a variable (e.g., choice).
5. SWITCH on choice:
 - Case 1: DISPLAY suitable application for 5 A fuse.
 - Case 2: DISPLAY suitable application for 10 A fuse.
 - Case 3: DISPLAY suitable application for 13 A fuse.
 - Case 4: DISPLAY suitable application for 20 A fuse.
 - Case 5: DISPLAY suitable application for 32 A fuse.
 - Default: DISPLAY 'Invalid choice. Please select 1 to 5.'
6. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY '--- Fuse Rating Selection Menu ---'
DISPLAY '1. 5 A Fuse'
DISPLAY '2. 10 A Fuse'
DISPLAY '3. 13 A Fuse'
DISPLAY '4. 20 A Fuse'
DISPLAY '5. 32 A Fuse'
DISPLAY 'Enter your choice (1-5): '
INPUT choice
SWITCH (choice)
CASE 1:
DISPLAY 'Application: Lighting circuits and small household appliances.'
BREAK
CASE 2:
DISPLAY 'Application: Electric showers and water heaters.'
BREAK
CASE 3:
DISPLAY 'Application: Standard 13 A socket outlets.'
BREAK
CASE 4:
DISPLAY 'Application: Immersion heaters and air conditioning units.'
BREAK
CASE 5:
DISPLAY 'Application: Cookers and high-power electrical equipment.'
BREAK
DEFAULT:
DISPLAY 'Invalid choice. Please select 1 to 5.'
END SWITCH
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
[DISPLAY Fuse Rating Menu]
|
v
/INPUT choice/
|
v
| | | | |
Case 1 Case 2 Case 3 Case 4 Case 5 Default
| | | | |
[5A App] [10A App] [13A App] [20A App] [32A App] [Invalid]
| | | | |
+-----+-----+-----+-----+-----+-----+
|
v
[STOP]

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Select 13 A Fuse

INPUT: Enter your choice (1-5): 3

OUTPUT: Application: Standard 13 A socket outlets.

Test Case 2 – Invalid Selection

INPUT: Enter your choice (1-5): 7

OUTPUT: Invalid choice. Please select 1 to 5.

QUESTION 5: Measurement Unit Conversion Menu

Required Control Statement: switch

1. ALGORITHM

1. START the programme.
2. DISPLAY the conversion menu:
 1. Convert volts to millivolts
 2. Convert amperes to milliamperes
 3. Convert kilowatts to watts
 4. Convert ohms to kilo-ohms
3. PROMPT the user to enter their menu choice.
4. READ and STORE choice into a variable.
5. PROMPT the user to enter the value to convert.
6. READ and STORE the value.
7. SWITCH on choice:
 - Case 1: $\text{result} = \text{value} \times 1000$; DISPLAY result in mV.
 - Case 2: $\text{result} = \text{value} \times 1000$; DISPLAY result in mA.
 - Case 3: $\text{result} = \text{value} \times 1000$; DISPLAY result in W.
 - Case 4: $\text{result} = \text{value} / 1000$; DISPLAY result in $k\Omega$.
 - Default: DISPLAY 'Invalid choice.'
8. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY '--- Unit Conversion Menu ---'
DISPLAY '1. Volts to Millivolts'
DISPLAY '2. Amperes to Milliamperes'
DISPLAY '3. Kilowatts to Watts'
DISPLAY '4. Ohms to Kilo-ohms'
DISPLAY 'Enter choice: '
INPUT choice
DISPLAY 'Enter value to convert: '
INPUT value
SWITCH (choice)
CASE 1:
    result = value * 1000
    DISPLAY value + ' V = ' + result + ' mV'
    BREAK
CASE 2:
    result = value * 1000
    DISPLAY value + ' A = ' + result + ' mA'
    BREAK
CASE 3:
    result = value * 1000
    DISPLAY value + ' kW = ' + result + ' W'
    BREAK
CASE 4:
    result = value / 1000
    DISPLAY value + ' Ohms = ' + result + ' kOhms'
    BREAK
DEFAULT:
    DISPLAY 'Invalid choice.'
END SWITCH
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
[DISPLAY Conversion Menu]
|
v
/INPUT choice/
|
v
/INPUT value/
|
v
| | | | |
Case 1 Case 2 Case 3 Case 4 Default
V->mV A->mA kW->W Ohms->kOhms [Invalid]
| | | | |
+---+---+---+---+---+---+
|
v
[STOP]

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Convert 5 kW to Watts

INPUT: Enter choice: 3 Enter value to convert: 5

OUTPUT: 5 kW = 5000 W

Test Case 2 – Convert 2500 Ohms to kilo-ohms

INPUT: Enter choice: 4 Enter value to convert: 2500

OUTPUT: 2500 Ohms = 2.5 kOhms

QUESTION 6: Transformer Ratio Calculator Menu

Required Control Statement: switch

1. ALGORITHM

1. START the programme.
2. DISPLAY the transformer calculation menu:
 1. Calculate turns ratio
 2. Calculate secondary voltage
 3. Calculate primary voltage
3. PROMPT the user to enter their menu choice.
4. READ and STORE choice into a variable.
5. SWITCH on choice:
Case 1:
 - a. INPUT primary turns (N1) and secondary turns (N2).
 - b. CALCULATE $\text{turnsRatio} = N1 / N2$.
 - c. DISPLAY turns ratio.Case 2:
 - a. INPUT primary voltage (V1), primary turns (N1), secondary turns (N2).
 - b. CALCULATE $V2 = V1 \times N2 / N1$.
 - c. DISPLAY secondary voltage in Volts.Case 3:
 - a. INPUT secondary voltage (V2), primary turns (N1), secondary turns (N2).
 - b. CALCULATE $V1 = V2 \times N1 / N2$.
 - c. DISPLAY primary voltage in Volts.Default: DISPLAY 'Invalid choice.'
6. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY '--- Transformer Ratio Calculator ---'
DISPLAY '1. Calculate Turns Ratio'
DISPLAY '2. Calculate Secondary Voltage'
DISPLAY '3. Calculate Primary Voltage'
DISPLAY 'Enter choice: '
INPUT choice
SWITCH (choice)
CASE 1:
INPUT N1, N2
turnsRatio = N1 / N2
DISPLAY 'Turns Ratio = ' + turnsRatio
BREAK
CASE 2:
INPUT V1, N1, N2
V2 = V1 * N2 / N1
DISPLAY 'Secondary Voltage = ' + V2 + ' V'
BREAK
CASE 3:
INPUT V2, N1, N2
V1 = V2 * N1 / N2
DISPLAY 'Primary Voltage = ' + V1 + ' V'
BREAK
DEFAULT:
DISPLAY 'Invalid choice.'
END SWITCH
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
[DISPLAY Transformer Menu]
|
v
/INPUT choice/
|
v
| | | |
Case 1 Case 2 Case 3 Default
| | | |
/INPUT N1,N2/ /INPUT V1,N1,N2/ /INPUT V2,N1,N2/ [Invalid]
| | |
|turnsRatio |V2 = V1*N2/N1| |V1 = V2*N1/N2|
= N1/N2| | |
| | |
[DISPLAY ratio] [DISPLAY V2 V] [DISPLAY V1 V]
| | | |
+-----+-----+-----+-----+
|
v
[STOP]

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Calculate Secondary Voltage

INPUT: Enter choice: 2 Enter primary voltage (V): 240 Enter primary turns: 1000
Enter secondary turns: 250

OUTPUT: Secondary Voltage = 60.00 V

Test Case 2 – Calculate Turns Ratio

INPUT: Enter choice: 1 Enter primary turns: 500 Enter secondary turns: 100

OUTPUT: Turns Ratio = 5.00

QUESTION 7: Total Power of Lighting Points

Required Control Statement: for loop

1. ALGORITHM

1. START the programme.
2. INITIALISE totalPower = 0.
3. FOR each lighting point from 1 to 12:
 - a. PROMPT the user to enter the wattage for the current lighting point.
 - b. READ and STORE the wattage into a variable.
 - c. ADD the wattage to totalPower.
4. END FOR loop.
5. DISPLAY the totalPower in Watts.
6. END the programme.

2. PSEUDOCODE

```
BEGIN
SET totalPower = 0
FOR i = 1 TO 12 DO
  DISPLAY 'Enter wattage for lighting point ' + i + ': '
  INPUT wattage
  totalPower = totalPower + wattage
END FOR
DISPLAY 'Total Lighting Load: ' + totalPower + ' W'
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
|SET totalPower = 0, i = 1|
|
v
--NO--> [DISPLAY 'Total Lighting Load: ' + totalPower + ' W'] --> [STOP]
| YES
|
v
/INPUT wattage for point i/
|
v
|totalPower = totalPower + wattage|
|
v
|i = i + 1|
|
v
(back to loop condition)
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Uniform 60 W Bulbs

INPUT: All 12 lighting points entered as: 60 W each

OUTPUT: Total Lighting Load: 720 W

Test Case 2 – Mixed Wattages

INPUT: Points 1-6: 40 W each; Points 7-12: 100 W each

OUTPUT: Total Lighting Load: 840 W

QUESTION 8: Average Current of Three-Phase Readings

Required Control Statement: for loop

1. ALGORITHM

1. START the programme.
2. FOR each set from 1 to 6:
 - a. PROMPT user to enter Line 1 current (I1) in Amperes.
 - b. PROMPT user to enter Line 2 current (I2) in Amperes.
 - c. PROMPT user to enter Line 3 current (I3) in Amperes.
 - d. CALCULATE $\text{averageCurrent} = (I1 + I2 + I3) / 3$.
 - e. DISPLAY the average current for this set.
3. END FOR loop.
4. END the programme.

2. PSEUDOCODE

```
BEGIN
FOR set = 1 TO 6 DO
  DISPLAY 'Set ' + set + ':'
  DISPLAY 'Enter Line 1 current (A): '
  INPUT I1
  DISPLAY 'Enter Line 2 current (A): '
  INPUT I2
  DISPLAY 'Enter Line 3 current (A): '
  INPUT I3
  averageCurrent = (I1 + I2 + I3) / 3
  DISPLAY 'Average current for Set ' + set + ': ' + averageCurrent + ' A'
END FOR
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
|SET set = 1|
|
v
--NO--> [STOP]
| YES
v
/INPUT I1, I2, I3/
|
v
|averageCurrent = (I1 + I2 + I3) / 3|
|
v
[DISPLAY averageCurrent for set]
|
v
|set = set + 1|
|
v
(back to loop condition)
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Set 1 Balanced Load

INPUT: Set 1: I1 = 30 A, I2 = 30 A, I3 = 30 A

OUTPUT: Average current for Set 1: 30.00 A

Test Case 2 – Set 2 Unbalanced Load

INPUT: Set 2: I1 = 20 A, I2 = 25 A, I3 = 33 A

OUTPUT: Average current for Set 2: 26.00 A

QUESTION 9: Count Passed Component Tests

Required Control Statement: for loop with if...else

1. ALGORITHM

1. START the programme.
2. INITIALISE passCount = 0 and failCount = 0.
3. FOR each component from 1 to 15:
 - a. PROMPT user to enter test result: 1 for pass, 0 for fail.
 - b. READ and STORE result.
 - c. IF result equals 1, INCREMENT passCount by 1.
 - d. ELSE IF result equals 0, INCREMENT failCount by 1.
 - e. ELSE DISPLAY: 'Invalid entry for component i.'
4. END FOR loop.
5. DISPLAY passCount (total passed components).
6. DISPLAY failCount (total failed components).
7. END the programme.

2. PSEUDOCODE

```
BEGIN
SET passCount = 0
SET failCount = 0
FOR i = 1 TO 15 DO
DISPLAY 'Component ' + i + ' result (1=Pass, 0=Fail): '
INPUT result
IF (result == 1) THEN
passCount = passCount + 1
ELSE IF (result == 0) THEN
failCount = failCount + 1
ELSE
DISPLAY 'Invalid entry for component ' + i
END IF
END FOR
DISPLAY 'Total Passed: ' + passCount
DISPLAY 'Total Failed: ' + failCount
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET passCount = 0, failCount = 0, i = 1|
|
v
--NO--> [DISPLAY passCount, failCount] --> [STOP]
| YES
v
/INPUT result/
|
v
--YES--> |passCount++|
| NO |
v |
--YES--> |failCount++|
| NO |
v |
[DISPLAY 'Invalid entry'] |
| |
+-----+-----+
|
v
|i = i + 1|
|
(back to loop)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – 10 Pass, 5 Fail

INPUT: 15 entries: 1,1,1,1,1,1,1,1,1,1,0,0,0,0,0

OUTPUT: Total Passed: 10 Total Failed: 5

Test Case 2 – All Pass

INPUT: 15 entries: all 1

OUTPUT: Total Passed: 15 Total Failed: 0

QUESTION 10: Calculate Total Energy for Appliances

Required Control Statement: for loop

1. ALGORITHM

1. START the programme.
2. INITIALISE totalEnergy = 0.
3. FOR each appliance from 1 to 8:
 - a. PROMPT user to enter the power rating in Watts.
 - b. PROMPT user to enter the usage time in hours.
 - c. CALCULATE energy = power × time.
 - d. DISPLAY individual energy for that appliance in Wh.
 - e. ADD energy to totalEnergy.
4. END FOR loop.
5. DISPLAY totalEnergy in watt-hours (Wh).
6. END the programme.

2. PSEUDOCODE

```
BEGIN
SET totalEnergy = 0
FOR i = 1 TO 8 DO
DISPLAY 'Appliance ' + i + ' - Enter power rating (W): '
INPUT power
DISPLAY 'Appliance ' + i + ' - Enter usage time (hours): '
INPUT time
energy = power * time
DISPLAY 'Energy for Appliance ' + i + ': ' + energy + ' Wh'
totalEnergy = totalEnergy + energy
END FOR
DISPLAY 'Total Energy Consumed: ' + totalEnergy + ' Wh'
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET totalEnergy = 0, i = 1|
|
v
--NO--> [DISPLAY totalEnergy + ' Wh'] --> [STOP]
| YES
v
/INPUT power (W), time (hrs)/
|
v
|energy = power * time|
|
v
[DISPLAY energy for appliance i]
|
v
|totalEnergy = totalEnergy + energy|
|
v
|i = i + 1|
|
v
(back to loop condition)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – 8 Appliances Mixed

INPUT: 8 appliances: 100W/3h, 200W/2h, 150W/4h, 50W/6h, 75W/5h, 300W/1h, 250W/2h, 400W/1h

OUTPUT: Total Energy Consumed: 3025 Wh

Test Case 2 – Uniform Appliances

INPUT: All 8 appliances: 500 W for 2 hours each

OUTPUT: Total Energy Consumed: 8000 Wh

QUESTION 11: Stop Motor Test When Temperature Is Unsafe

Required Control Statement: while loop

1. ALGORITHM

1. START the programme.
2. PROMPT the user to enter the first motor temperature reading in °C.
3. READ and STORE temperature into a variable (e.g., temp).
4. WHILE temp is less than or equal to 90°C:
 - a. DISPLAY: 'Temperature acceptable. Continuing test.'
 - b. PROMPT user to enter the next motor temperature reading.
 - c. READ and STORE new temperature value.
5. END WHILE loop.
6. DISPLAY: 'Motor temperature unsafe. Stop motor test.'
7. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY 'Enter motor temperature reading (°C): '
INPUT temp
WHILE (temp <= 90) DO
  DISPLAY 'Temperature acceptable. Continuing test.'
  DISPLAY 'Enter next motor temperature reading (°C): '
  INPUT temp
END WHILE
DISPLAY 'Motor temperature unsafe. Stop motor test.'
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
/INPUT temp/
|
v
--NO--> [DISPLAY 'Motor temperature unsafe. Stop motor test.'] --> [STOP]
| YES
v
[DISPLAY 'Temperature acceptable. Continuing test.']
|
v
/INPUT next temp/
|
v
(back to loop condition)
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Triggers After Safe Readings

INPUT: Readings: 75, 82, 88, 95

OUTPUT: Temperature acceptable. (x3) Motor temperature unsafe. Stop motor test.

Test Case 2 – Immediate Unsafe Reading

INPUT: Reading: 100

OUTPUT: Motor temperature unsafe. Stop motor test.

QUESTION 12: Solar Panel Voltage Tracker

Required Control Statement: while loop

1. ALGORITHM

1. START the programme.
2. PROMPT the user to enter the first solar panel voltage reading in Volts.
3. READ and STORE voltage into a variable (e.g., voltage).
4. WHILE voltage is greater than or equal to 18 V:
 - a. DISPLAY: 'Voltage acceptable. System operating.'
 - b. PROMPT user to enter the next voltage reading.
 - c. READ and STORE new voltage value.
5. END WHILE loop.
6. DISPLAY: 'Solar panel voltage below operating level.'
7. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY 'Enter solar panel voltage reading (V): '
INPUT voltage
WHILE (voltage >= 18) DO
DISPLAY 'Voltage acceptable. System operating.'
DISPLAY 'Enter next solar panel voltage reading (V): '
INPUT voltage
END WHILE
DISPLAY 'Solar panel voltage below operating level.'
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
/INPUT voltage/
|
v
= 18 V?>--NO--> [DISPLAY 'Solar panel voltage below operating level.'] --> [STOP]
| YES
v
[DISPLAY 'Voltage acceptable. System operating.']
|
v
/INPUT next voltage/
|
v
(back to loop condition)
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Drops Below Threshold

INPUT: Readings: 22.5, 20.0, 18.0, 15.3

OUTPUT: Voltage acceptable. (x3) Solar panel voltage below operating level.

Test Case 2 – Immediate Drop

INPUT: Reading: 12.0

OUTPUT: Solar panel voltage below operating level.

QUESTION 13: Earth Resistance Test

Required Control Statement: while loop

1. ALGORITHM

1. START the programme.
2. PROMPT the user to enter the first earth resistance reading in Ohms.
3. READ and STORE resistance into a variable (e.g., resistance).
4. WHILE resistance is less than or equal to 5 Ohms:
 - a. DISPLAY: 'Earth resistance acceptable.'
 - b. PROMPT user to enter the next resistance reading.
 - c. READ and STORE new resistance value.
5. END WHILE loop.
6. DISPLAY: 'Earth resistance too high. Improve earthing system.'
7. END the programme.

2. PSEUDOCODE

```
BEGIN
DISPLAY 'Enter earth resistance reading (Ohms): '
INPUT resistance
WHILE (resistance <= 5) DO
  DISPLAY 'Earth resistance acceptable.'
  DISPLAY 'Enter next earth resistance reading (Ohms): '
  INPUT resistance
END WHILE
DISPLAY 'Earth resistance too high. Improve earthing system.'
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
/INPUT resistance/
|
v
--NO--> [DISPLAY 'Earth resistance too high.'] --> [STOP]
| YES
v
[DISPLAY 'Earth resistance acceptable.']
|
v
/INPUT next resistance/
|
v
(back to loop condition)
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Resistance Rises Above Limit

INPUT: Readings: 2.5, 4.0, 5.0, 6.8

OUTPUT: Earth resistance acceptable. (x3) Earth resistance too high. Improve earthing system.

Test Case 2 – Immediate High Resistance

INPUT: Reading: 8.0

OUTPUT: Earth resistance too high. Improve earthing system.

QUESTION 14: Laboratory Login Attempt

Required Control Statement: while loop with a counter

1. ALGORITHM

1. START the programme.
2. SET attempts = 0 and correctCode = 'BEE208'.
3. WHILE attempts is less than 3:
 - a. INCREMENT attempts by 1.
 - b. PROMPT user to enter the laboratory access code.
 - c. READ and STORE the entered code.
 - d. IF enteredCode equals correctCode:
 - i. DISPLAY: 'Access granted.'
 - ii. EXIT the loop (break).
 - e. ELSE:
 - i. DISPLAY: 'Incorrect code. Attempt ' + attempts + ' of 3.'
4. END WHILE loop.
5. IF attempts equals 3 AND access was not granted:
 - a. DISPLAY: 'Access denied. Maximum attempts reached.'
6. END the programme.

2. PSEUDOCODE

```
BEGIN
SET correctCode = 'BEE208'
SET attempts = 0
SET accessGranted = FALSE
WHILE (attempts < 3) DO
  attempts = attempts + 1
  DISPLAY 'Enter laboratory access code: '
  INPUT enteredCode
  IF (enteredCode == correctCode) THEN
    DISPLAY 'Access granted.'
    SET accessGranted = TRUE
    BREAK
  ELSE
    DISPLAY 'Incorrect code. Attempt ' + attempts + ' of 3.'
  END IF
END WHILE
IF (accessGranted == FALSE) THEN
  DISPLAY 'Access denied. Maximum attempts reached.'
END IF
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET attempts = 0, accessGranted = FALSE|
|
v
--NO--> --NO--> [DISPLAY 'Access denied.'] --> [STOP]
| YES |YES
v v
|attempts++| [STOP]
|
v
/INPUT enteredCode/
|
v
--YES--> [DISPLAY 'Access granted.'] --> [STOP]
| NO
v
[DISPLAY 'Incorrect code. Attempt X of 3.']
|
v
(back to loop condition)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Correct Code on Second Attempt

INPUT: Attempt 1: WRONG123 Attempt 2: BEE208

OUTPUT: Incorrect code. Attempt 1 of 3. Access granted.

Test Case 2 – All Three Attempts Failed

INPUT: Attempt 1: ABC Attempt 2: 123 Attempt 3: XYZ

OUTPUT: Incorrect (x3) Access denied. Maximum attempts reached.

QUESTION 15: Repeated Ohm's Law Calculator

Required Control Statement: do...while loop

1. ALGORITHM

1. START the programme.
2. DO:
 - a. PROMPT the user to enter voltage in Volts.
 - b. READ and STORE voltage.
 - c. PROMPT the user to enter current in Amperes.
 - d. READ and STORE current.
 - e. CHECK: If current is not zero:
 - i. CALCULATE $\text{resistance} = \text{voltage} / \text{current}$.
 - ii. DISPLAY resistance in Ohms.
 - ELSE:
 - i. DISPLAY: 'Error: Current cannot be zero.'
 - f. PROMPT user: 'Do you want to continue? (Y/N)'
 - g. READ and STORE choice.
3. WHILE choice equals 'Y'.
4. END the programme.

2. PSEUDOCODE

```
BEGIN
DO
  DISPLAY 'Enter voltage (V): '
  INPUT voltage
  DISPLAY 'Enter current (A): '
  INPUT current
  IF (current != 0) THEN
    resistance = voltage / current
    DISPLAY 'Resistance = ' + resistance + ' Ohms'
  ELSE
    DISPLAY 'Error: Current cannot be zero.'
  END IF
  DISPLAY 'Continue? (Y/N): '
  INPUT continueChoice
  WHILE (continueChoice == 'Y')
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
/INPUT voltage, current/ <-- (D0 block begins here)
|
v
--YES--> |resistance = voltage / current|
| NO |
v v
[DISPLAY 'Error: Current [DISPLAY resistance + ' Ohms']
cannot be zero.']]
| |
+-----+-----+
|
v
/INPUT continueChoice/
|
v
--YES--> (back to D0 block)
| NO
v
[STOP]

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Single Calculation

INPUT: Voltage: 230 V Current: 5 A Continue? N

OUTPUT: Resistance = 46.00 Ohms

Test Case 2 – Two Calculations

INPUT: Voltage: 12 V, Current: 0.5 A, Continue? Y Voltage: 120 V, Current: 4 A, Continue? N

OUTPUT: Resistance = 24.00 Ohms Resistance = 30.00 Ohms

QUESTION 16: Repeated DC Power Calculator

Required Control Statement: do...while loop

1. ALGORITHM

1. START the programme.
2. DO:
 - a. PROMPT user to enter DC voltage in Volts.
 - b. READ and STORE voltage.
 - c. PROMPT user to enter DC current in Amperes.
 - d. READ and STORE current.
 - e. CALCULATE $\text{power} = \text{voltage} \times \text{current}$.
 - f. DISPLAY power in Watts.
 - g. PROMPT: 'Perform another calculation? (Y/N)'
 - h. READ and STORE choice.
3. WHILE choice equals 'Y'.
4. END the programme.

2. PSEUDOCODE

```
BEGIN
DO
DISPLAY 'Enter DC voltage (V): '
INPUT voltage
DISPLAY 'Enter DC current (A): '
INPUT current
power = voltage * current
DISPLAY 'DC Power = ' + power + ' W'
DISPLAY 'Perform another calculation? (Y/N): '
INPUT continueChoice
WHILE (continueChoice == 'Y')
END
```

3. FLOWCHART TEXT-MAP

```
[START]
|
v
/INPUT voltage, current/ <-- (DO block begins here)
|
v
|power = voltage * current|
|
v
[DISPLAY 'DC Power = ' + power + ' W']
|
v
/INPUT continueChoice/
|
v
--YES--> (back to DO block)
| NO
v
[STOP]
```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 - Single Calculation

INPUT: Voltage: 24 V Current: 3 A Continue? N

OUTPUT: DC Power = 72.00 W

Test Case 2 - Two Calculations

INPUT: V=12, I=2, Y; V=48, I=5, N

OUTPUT: DC Power = 24.00 W DC Power = 240.00 W

QUESTION 17: Stop Insulation Test Using Break

Required Control Statement: for loop with break

1. ALGORITHM

1. START the programme.
2. FOR each reading from 1 to 10:
 - a. PROMPT user to enter insulation resistance reading in megaohms.
 - b. READ and STORE the reading.
 - c. IF reading is less than 1 megaohm:
 - i. DISPLAY: 'Insulation failure detected. Test stopped.'
 - ii. BREAK out of the loop.
 - d. ELSE:
 - i. DISPLAY: 'Insulation reading acceptable.'
3. END FOR loop.
4. END the programme.

2. PSEUDOCODE

```
BEGIN
FOR i = 1 TO 10 DO
  DISPLAY 'Enter insulation resistance reading ' + i + ' (MΩ): '
  INPUT reading
  IF (reading < 1) THEN
    DISPLAY 'Insulation failure detected. Test stopped.'
    BREAK
  ELSE
    DISPLAY 'Insulation reading acceptable.'
  END IF
END FOR
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET i = 1|
|
v
--NO--> [STOP]
| YES
v
/INPUT reading (MΩ)/
|
v
--YES--> [DISPLAY 'Insulation failure. Test stopped.'] --> [STOP]
| NO
v
[DISPLAY 'Insulation reading acceptable.']
|
v
|i = i + 1|
|
v
(back to loop condition)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Failure on 4th Reading

INPUT: Readings: 5.0, 3.2, 2.8, 0.6

OUTPUT: Acceptable (x3) Insulation failure detected. Test stopped.

Test Case 2 – All 10 Readings Acceptable

INPUT: Readings: 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0

OUTPUT: Insulation reading acceptable. (x10)

QUESTION 18: Stop Load Test When Limit Is Reached

Required Control Statement: for loop with break

1. ALGORITHM

1. START the programme.
2. INITIALISE totalLoad = 0.
3. FOR each appliance from 1 to 20:
 - a. PROMPT user to enter power rating of the appliance in Watts.
 - b. READ and STORE powerRating.
 - c. ADD powerRating to totalLoad.
 - d. IF totalLoad is greater than 5000 W:
 - i. DISPLAY: 'Maximum load exceeded. Stop adding appliances.'
 - ii. BREAK out of the loop.
4. END FOR loop.
5. DISPLAY the final totalLoad in Watts.
6. END the programme.

2. PSEUDOCODE

```
BEGIN
SET totalLoad = 0
FOR i = 1 TO 20 DO
    DISPLAY 'Enter power rating of appliance ' + i + ' (W): '
    INPUT powerRating
    totalLoad = totalLoad + powerRating
    IF (totalLoad > 5000) THEN
        DISPLAY 'Maximum load exceeded. Stop adding appliances.'
        BREAK
    END IF
END FOR
DISPLAY 'Final Total Load: ' + totalLoad + ' W'
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET totalLoad = 0, i = 1|
|
v
--NO--> [DISPLAY 'Final Total Load: ' + totalLoad] --> [STOP]
| YES
v
/INPUT powerRating/
|
v
|totalLoad = totalLoad + powerRating|
|
v
5000 W?>--YES--> [DISPLAY 'Maximum load exceeded.'] --> [DISPLAY totalLoad] -->
[STOP]
| NO
v
|i = i + 1|
|
v
(back to loop condition)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Limit Exceeded on 6th Appliance

INPUT: Appliances: 1000, 1000, 1000, 1000, 1000, 500

OUTPUT: Maximum load exceeded. Stop adding appliances. Final Total Load: 5500 W

Test Case 2 – Completes All 20 Under Limit

INPUT: All 20 appliances: 200 W each

OUTPUT: Final Total Load: 4000 W

QUESTION 19: Skip Faulty Sensor Readings

Required Control Statement: for loop with continue

1. ALGORITHM

1. START the programme.
2. INITIALISE sum = 0 and validCount = 0.
3. FOR each reading from 1 to 12:
 - a. PROMPT user to enter sensor reading.
 - b. READ and STORE reading.
 - c. IF reading equals 999 (faulty indicator):
 - i. DISPLAY: 'Faulty sensor reading detected. Skipping.'
 - ii. CONTINUE to next iteration (skip this reading).
 - d. ADD reading to sum.
 - e. INCREMENT validCount by 1.
4. END FOR loop.
5. IF validCount > 0:
 - a. CALCULATE average = sum / validCount.
 - b. DISPLAY average of valid readings.
6. ELSE: DISPLAY 'No valid readings available.'
7. END the programme.

2. PSEUDOCODE

```
BEGIN
SET sum = 0
SET validCount = 0
FOR i = 1 TO 12 DO
    DISPLAY 'Enter sensor reading ' + i + ': '
    INPUT reading
    IF (reading == 999) THEN
        DISPLAY 'Faulty sensor reading detected. Skipping.'
        CONTINUE
    END IF
    sum = sum + reading
    validCount = validCount + 1
END FOR
IF (validCount > 0) THEN
    average = sum / validCount
    DISPLAY 'Average of valid readings: ' + average
ELSE
    DISPLAY 'No valid readings available.'
END IF
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET sum = 0, validCount = 0, i = 1|
|
v
--NO--> 0?> --YES--> |average = sum/validCount| --> [DISPLAY average] --> [STOP]
| YES |NO
v [DISPLAY 'No valid readings'] --> [STOP]
/INPUT reading/
|
v
--YES--> [DISPLAY 'Faulty. Skipping.'] --> |i++| --> (back to loop)
| NO
v
|sum = sum + reading|
|
v
|validCount++|
|
v
|i = i + 1|
|
v
(back to loop condition)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Two Faulty Readings

INPUT: 10 valid readings of 50 each + 2 readings of 999

OUTPUT: Faulty sensor reading detected. Skipping. (x2) Average of valid readings: 50.00

Test Case 2 – All Valid Readings

INPUT: 12 readings: 100, 120, 110, 130, 105, 115, 125, 108, 112, 118, 122, 116

OUTPUT: Average of valid readings: 115.08

QUESTION 20: Skip Negative Power Readings

Required Control Statement: for loop with continue

1. ALGORITHM

1. START the programme.
2. INITIALISE totalPower = 0.
3. FOR each reading from 1 to 10:
 - a. PROMPT user to enter a power reading.
 - b. READ and STORE the reading.
 - c. IF reading is negative (less than 0):
 - i. DISPLAY: 'Invalid reading skipped.'
 - ii. CONTINUE to next iteration.
 - d. ADD reading to totalPower.
4. END FOR loop.
5. DISPLAY totalPower in Watts.
6. END the programme.

2. PSEUDOCODE

```
BEGIN
SET totalPower = 0
FOR i = 1 TO 10 DO
  DISPLAY 'Enter power reading ' + i + ' (W): '
  INPUT reading
  IF (reading < 0) THEN
    DISPLAY 'Invalid reading skipped.'
    CONTINUE
  END IF
  totalPower = totalPower + reading
END FOR
DISPLAY 'Total Valid Power: ' + totalPower + ' W'
END
```

3. FLOWCHART TEXT-MAP

```

[START]
|
v
|SET totalPower = 0, i = 1|
|
v
--NO--> [DISPLAY 'Total Valid Power: ' + totalPower + ' W'] --> [STOP]
| YES
v
/INPUT reading/
|
v
--YES--> [DISPLAY 'Invalid reading skipped.'] --> |i++| --> (back to loop)
| NO
v
|totalPower = totalPower + reading|
|
v
|i = i + 1|
|
v
(back to loop condition)

```

4. TEST RESULTS & OUTPUT EVIDENCE

Test Case 1 – Three Negative Readings

INPUT: Readings: 500, -20, 300, -5, 200, 400, -10, 150, 250, 100

OUTPUT: Invalid reading skipped. (x3) Total Valid Power: 1900 W

Test Case 2 – All Valid Readings

INPUT: Readings: 100, 200, 300, 150, 250, 180, 220, 130, 170, 200

OUTPUT: Total Valid Power: 1900 W